Quiz #5

Kabir Kanha Arora
CSD 303 - Theory of Computation
Shiv Nadar University

April 21, 2020

Question 1

Many questions about CFLs/CFGs are not decidable. We will see some of them when we discuss undecidability. But some questions can be decided. For each of the following, give an algorithm that will give a yes or no answer.

(a) Given a CFG G is L(G) empty or not.

(b) Given a CFG G is L(G) finite or not.

(c) Given a variable X in a CFG G is X generating or not. A variable X is generating if it is part of some derivation that derives a terminal string starting from S (the start symbol).

(a) If we cannot derive any string of terminals from the given grammar G, then L(G) is said to be empty. The algorithm for this can be given as follows:

- Remove all the useless symbols from the grammar.
- A useless symbol is one that does not derive any string of terminals
- If the start symbol is found to be useless, the the language is empty. Otherwise it is not.

As an example to show this algorithm, consider the following grammar G:

$$S \to XY$$
$$X \to AX$$
$$X \to AA$$
$$A \to a$$
$$Y \to BY$$
$$Y \to BB$$
$$B \to b$$

This grammar can now be written as:

$$S \to aabb$$
$$X \to aX$$
$$X \to aa$$
$$A \to a$$
$$Y \to bY$$
$$Y \to bb$$
$$B \to b$$

Clearly, the start symbol S generates at least one string and hence the start symbol is useful. Thus, the language generated by the given grammar is non-empty i.e., $L(G) \neq \phi$

Note that, if the language generated by the CFG contains just a null string, then it is considered as non-empty.

(b) We can convert the grammar G to a grammar G^1, which is in CNF. Now, this grammar can easily be represented by a directed graph, since it only has transitions of the type $A \to BC$ and $A \to a$. The CGF can be thought of as an adjacency list of the graph. We observe that iff the graph contains a directed cycle, $L(G)$ is infinite, otherwise it is finite.

So the algorithm has to perform the following tasks:

- Represent the CFG in CNF as a directed graph.
- Check for cycles in the graph.
- If a cycle is found, then the graph is infinite. Otherwise, it is finite.

(c) Let us build on our premise for part (b). So that means we have the language in CNF and it can be represented as a directed graph. The variable X will be generating $\Leftrightarrow$ $\exists$ a path from the start symbol S to X. This path can be found using BFS starting from S. If a path does not exist, X is non-generating. This is intuitive, because if X cannot be derived from S, then it is certainly non-generaing.

Question 2

Design Turing machines (TMs) using the transition diagram or machine notation for each of the following. Where needed the initial and final position of the read/write head is shown with an underline.

(a) $S_{L\gamma}$ **- A TM that shifts a string** w **between two** γ **symbols where** $\gamma \notin w$ **one square to the left.** $S_0, \underline{\$}u\gamma w\gamma\not{b}... \to \$uw\gamma\underline{\not{b}}...$

(b) E **- Erase TM erases the tape.** $\underline{\$}w\not{b}... \rightarrow \underline{\$}\not{b}...$ **where** $\not{b} \notin w$

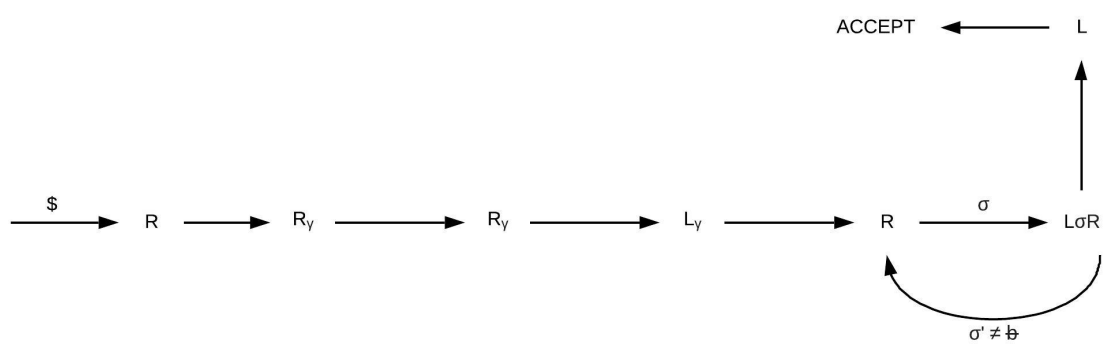
(c) A matching TM M_γ**. This TM starts with the input tape:** $\underline{\$}\gamma w_1 \gamma w_2 \gamma u\not{b}...$ **where** $\gamma \notin w_1, w2$**.**

If $w_1 = w_2$ **it writes** Y **on the third** γ **and stops it else it writes an** N **in the same position and stops. So, if** $w_1 = w_2$**, the final tape looks like** $\underline{\$}\gamma w_1 \gamma w_2 Y u\not{b}...$ **else it looks like** $\underline{\$}\gamma w_1 \gamma w_2 N u\not{b}...$

(d) Distinctness TM D**. D's input tape looks like:** $\underline{\$}\#w_1\#w_2\#...\#w_k\#\not{b}...$

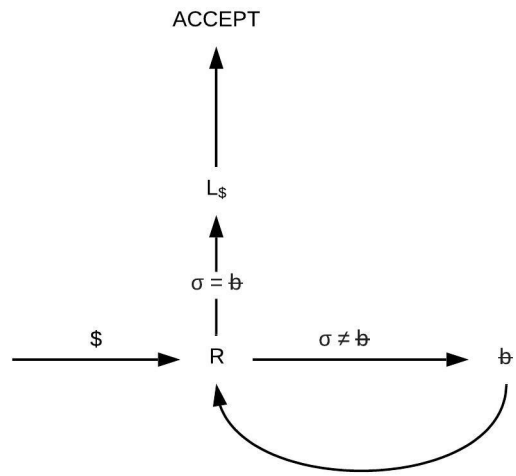
D **accepts if** $w_i \neq w_j$ **for** $i \neq j$ **else rejects. That is, it accepts if all** k **strings are distinct and rejects if any two are equal. You can use the TM in part (c) or invent another TM or set of TMs that will help you to build** D**.**

(a) It is unclear what happens to the string to the right of the second γ, so instead of assuming that it is the last character in the string, I design the Turing machine in such a way that if there is any string remaining after the second γ, then it is also shifted left by one block.



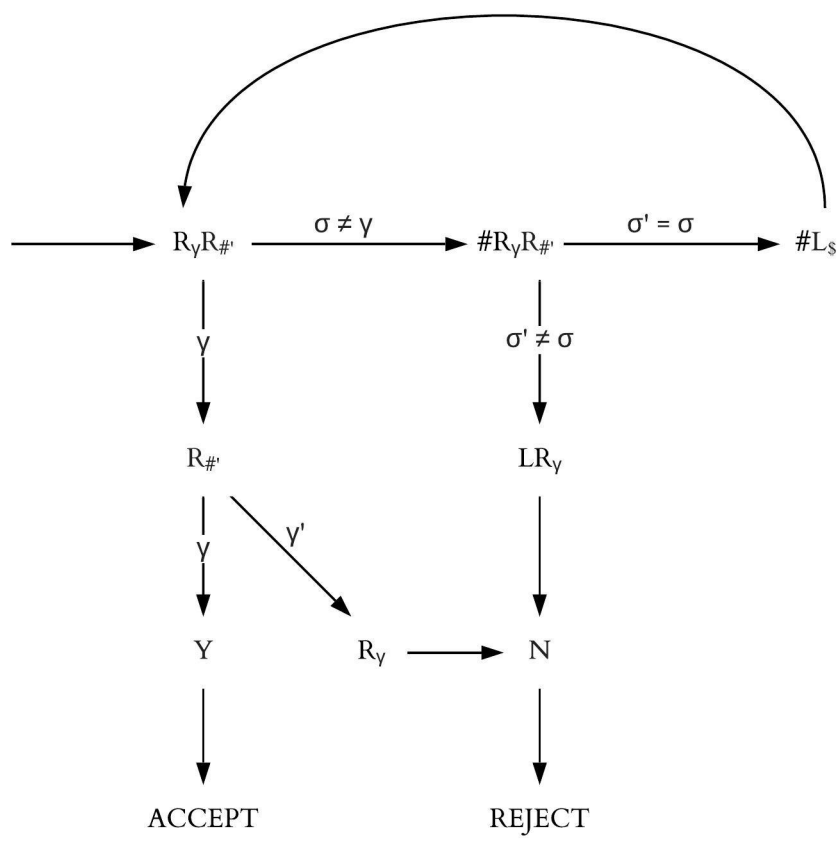
Note that we have the redundant looking R_γ and L_γ. They are used to check that the string indeed has two γ symbols. The L at the end serves to bring the read/write head to the desired location.

(b) The TM E can be represented as follows:

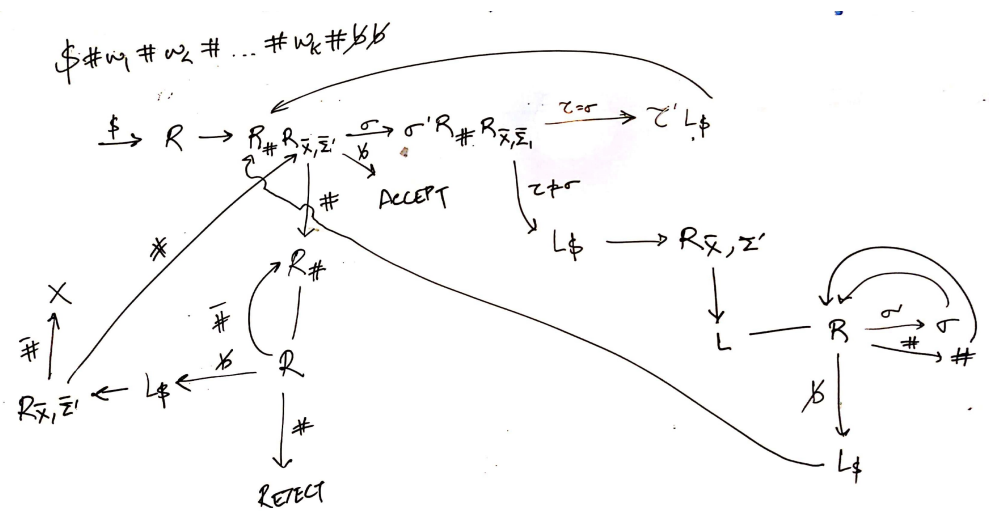


The $L_\$$ at the end ensures that the read/write head finishes at the desired location.

(c) The TM M_γ can be represented as follows:



(d) The algorithms for this is a fairly straightforward one. We start by comparing w_1 with w_2, $w_3 ... w_k$. If the strings match at any point, we reject the input. Otherwise, we move on to compare w_2 with w_3, $w_4 ... w_k$, and continue this process till we have compared all pairs of strings. If there is no match till the end, we accept the input.

For each $\sigma \in \Sigma$, I introduce a symbol $\sigma' \notin \Sigma$ which is used to indicate that the symbol has already been read. This will help us easily know where to read from as well as make is easier to restore the string when needed.

A rough sketch of the given Turing Machine could be as follows: